



**CMC UNIVERSITY**

Aspire to Inspire the Digital World

# Machine Learning and Data Mining

Faculty of Information and Communication Technology

## Attention and Transformer Mechanism

*In psychology, attention is the cognitive process of selectively concentrating on one or a few things while ignoring others.*

## Learning Objectives

- Understand the bottleneck of traditional Seq2Seq models (RNNs/LSTMs).
- Master the mathematical framework of Attention (Queries, Keys, Values).
- Deep dive into the Transformer architecture (Vaswani et al., 2017).
- Explain the role of Multi-Head Attention and Positional Encoding.

## From Seq2Seq to Attention

- Traditional Seq2Seq: Encoder compresses input into a fixed-length vector.
- The Bottleneck: Long sequences lose information; the "fixed-length" becomes a constraint.
- Solution: Attention allows the decoder to "look back" at specific parts of the input sequence.

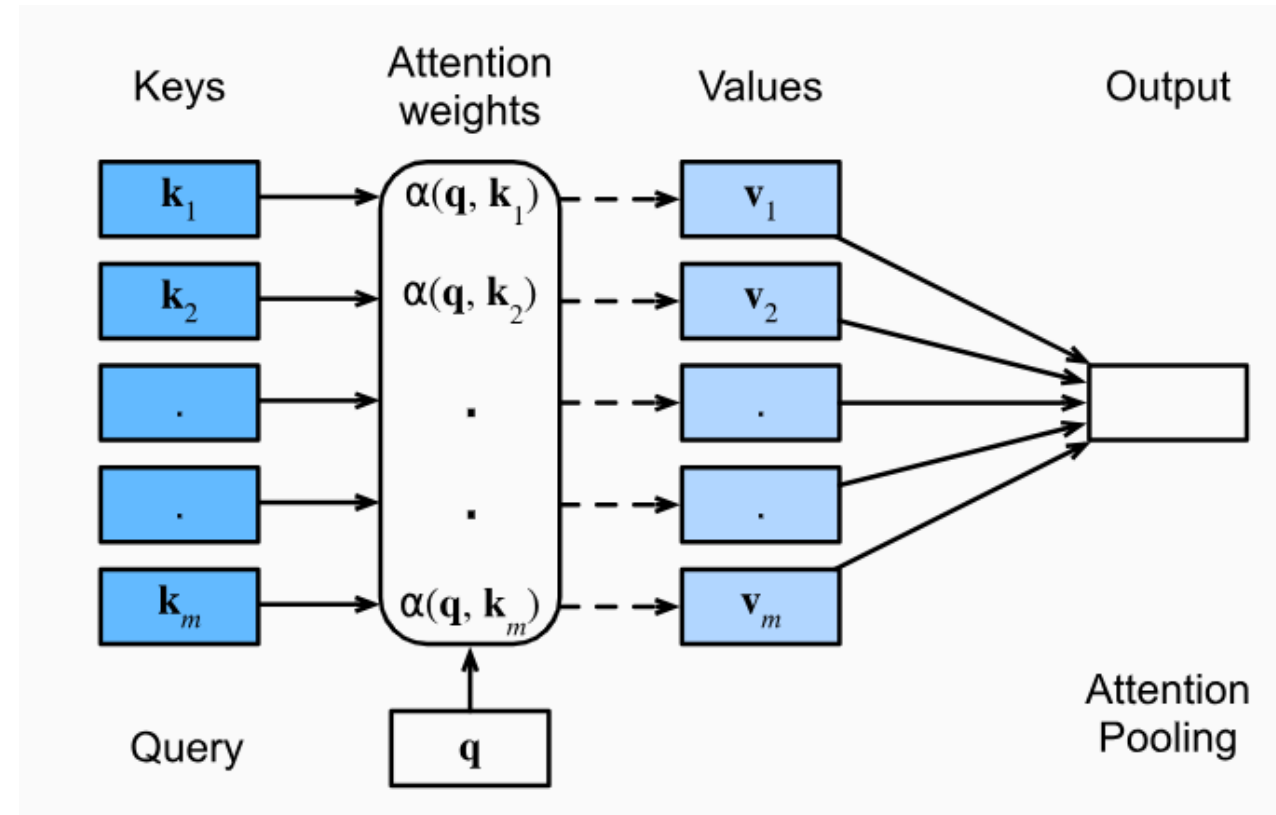
## Human Intuition of Attention

- When reading, we focus on specific words to understand context.
- In computer vision, we focus on salient objects in an image.

*The attention mechanism emerged as an improvement over the encoder decoder-based neural machine translation system in natural language processing (NLP). Later, this mechanism, or its variants, was used in other applications, including computer vision, speech processing, education, etc*

## General Attention Framework (Q, K, V)

- Query ( $Q$ ): What we are looking for.
- Key ( $K$ ): Description of the information available.
- Value ( $V$ ): The actual information to be retrieved.
- Attention output is a weighted sum of Values based on the similarity between Query and Keys.





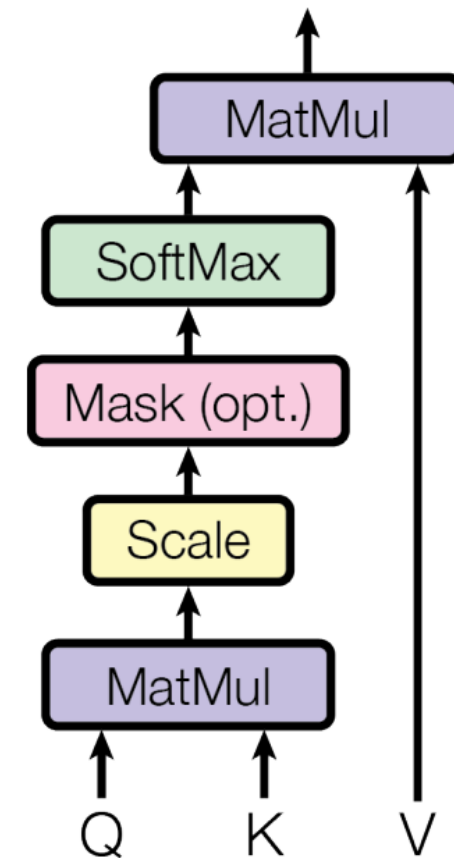
## Mathematical Definition

- $Attention(Q, K, V) = softmax(\alpha(Q, K)) V$
- Where  $\alpha(Q, K)$  is a scoring function.

## Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

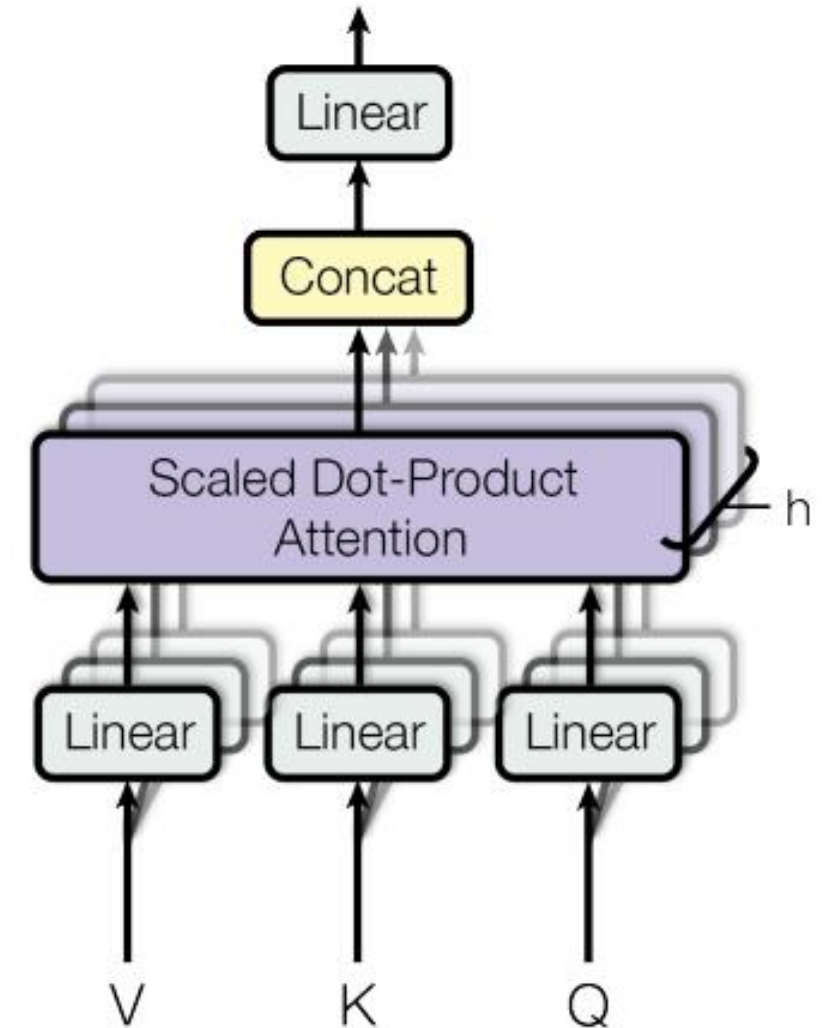
$\sqrt{d_k}$  is the scaling factor to prevent large values in the dot product.



Source: [Attention Is All You Need](#)

## Multi-Head Attention

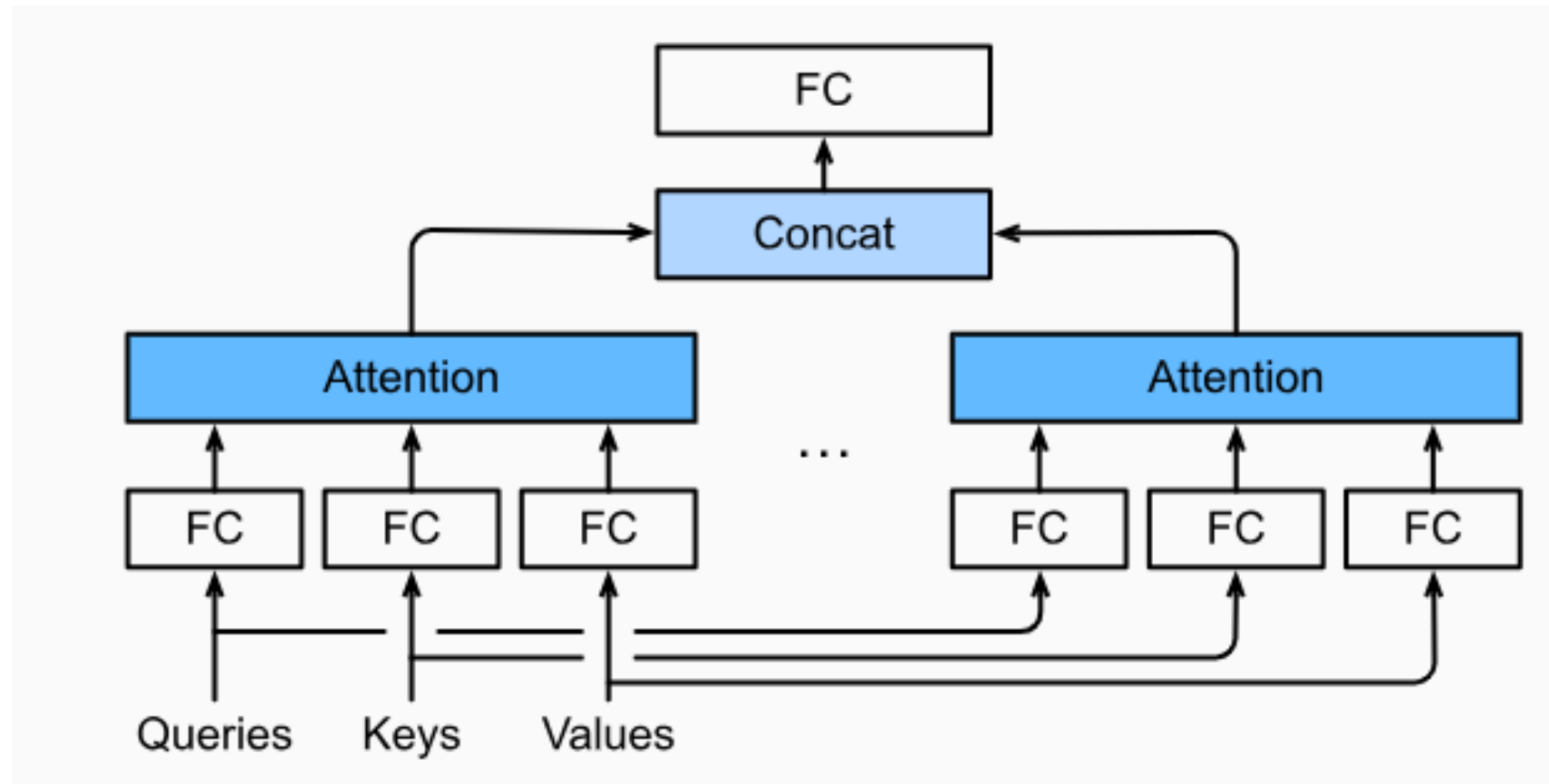
- Instead of one attention function, perform attention  $h$  times in parallel.
- Allows the model to jointly attend to information from different representation subspaces at different positions.





## Multi-Head Attention

- Linear projections for  $Q$ ,  $K$ ,  $V$ .
- Parallel attention heads.
- Concatenation of results followed by a final Linear layer.



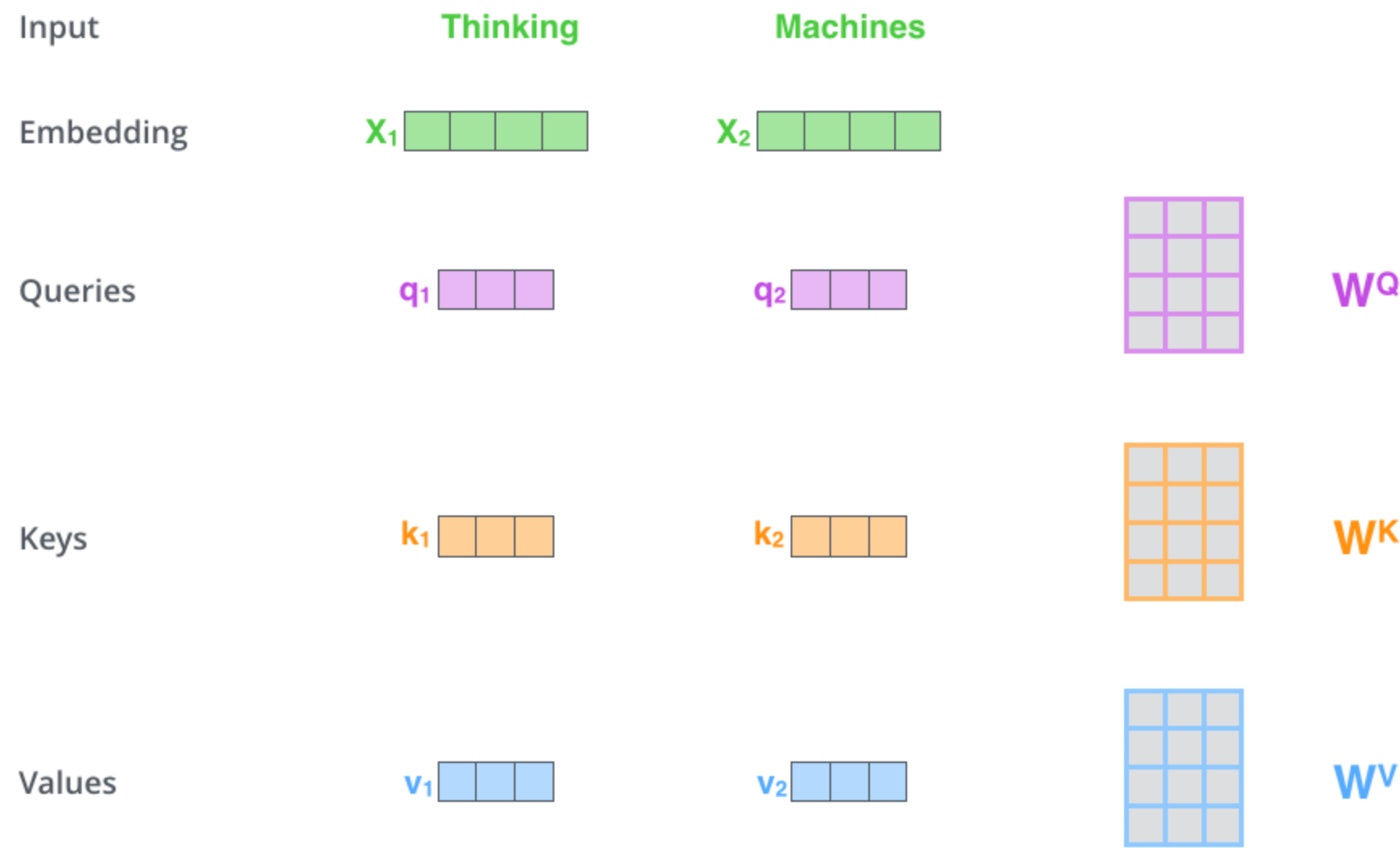
## Multi-Head Attention

- Linear projections for  $Q$ ,  $K$ ,  $V$ .
- Parallel attention heads.
- Concatenation of results followed by a final Linear layer.

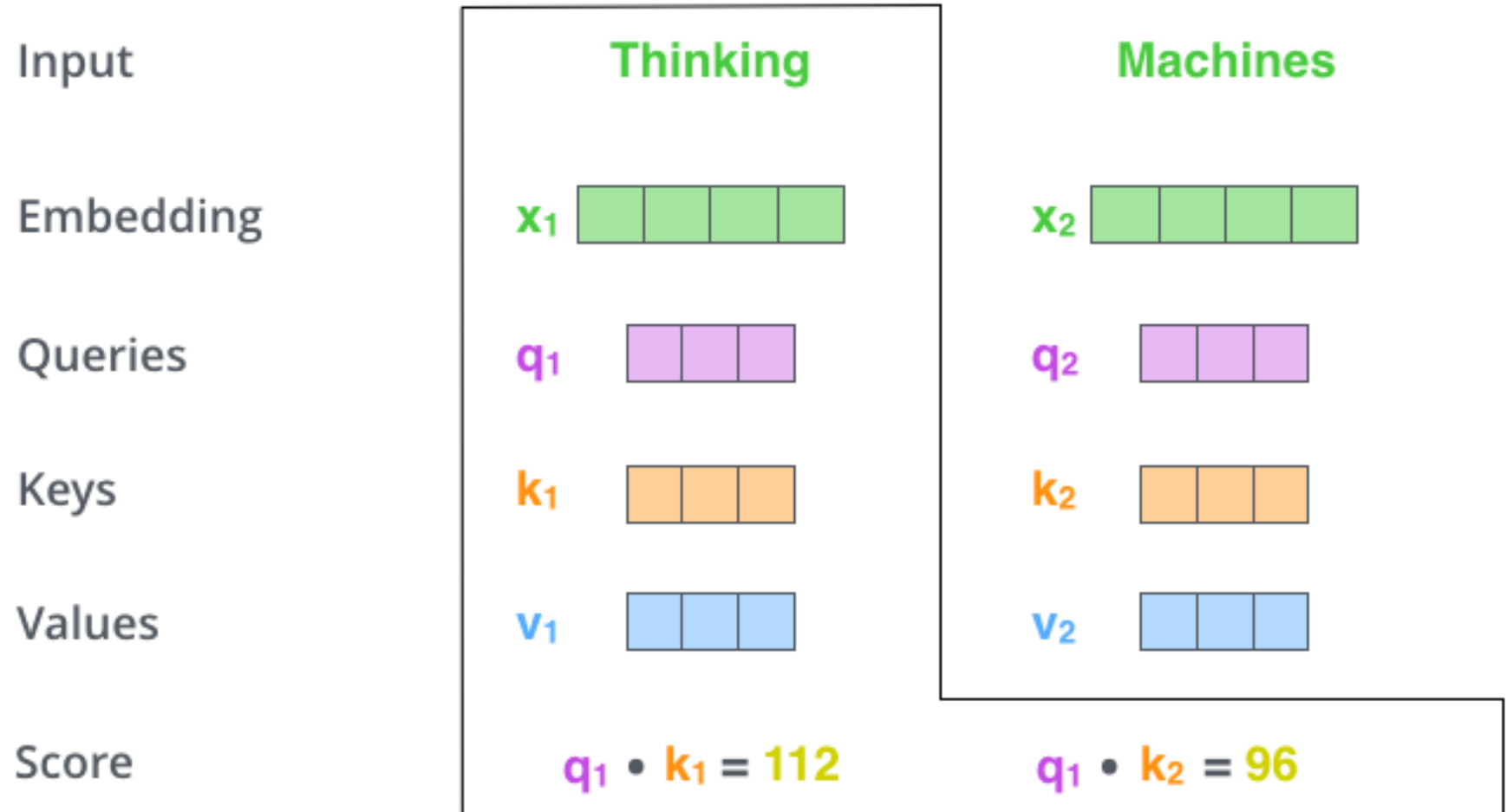
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where  $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

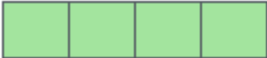
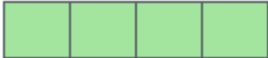
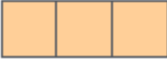
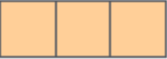
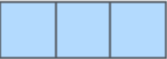
The first step in calculating self-attention is to create three vectors from each of the encoder's input vectors



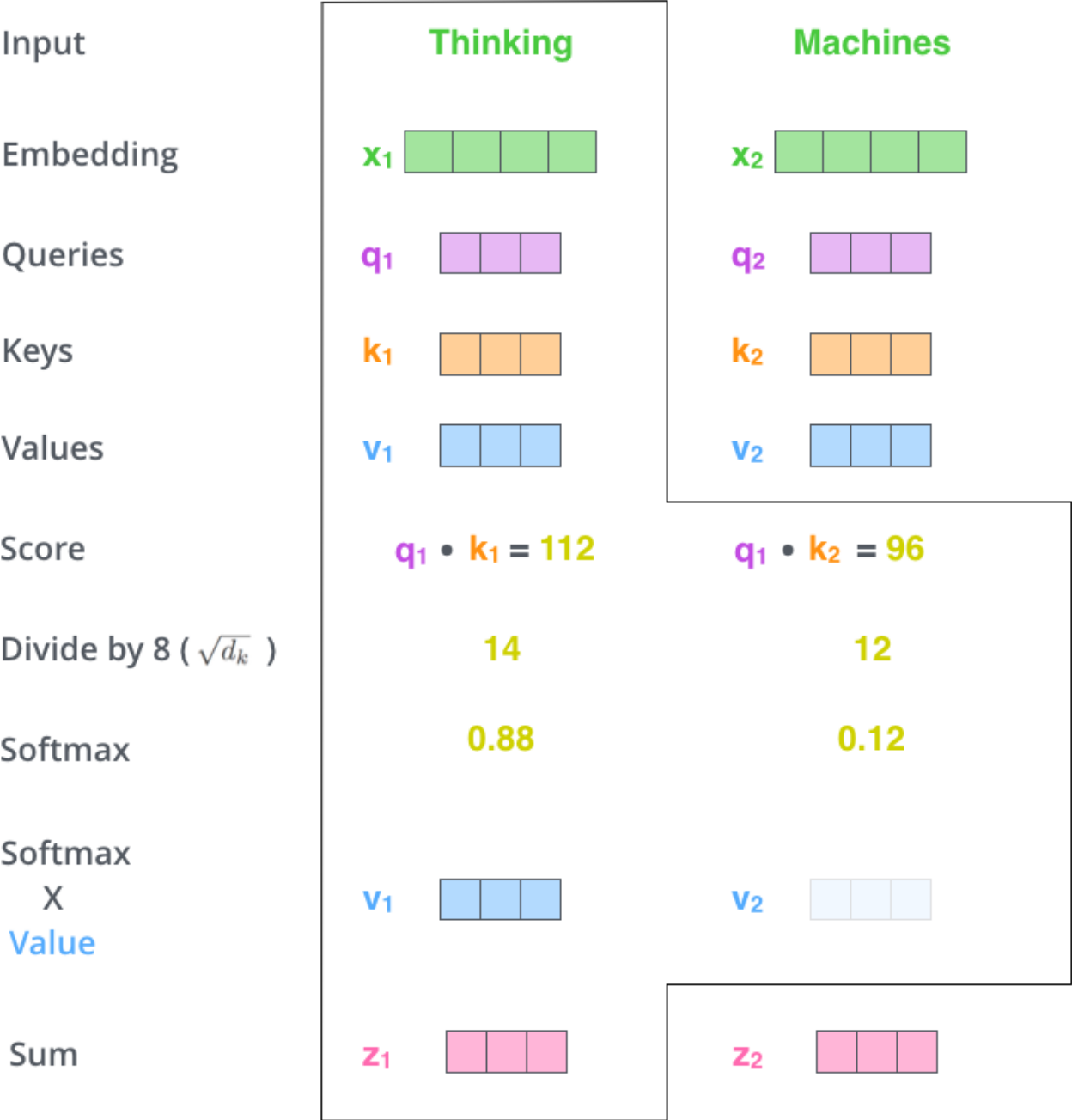
The second step in calculating self-attention is to calculate a score by taking the dot product of the query vector with the key vector



The third and fourth steps are to divide the scores by 8 (the square root of the dimension of the key vectors used in the paper – 64

| Input                        | Thinking                                                                                  | Machines                                                                                  |
|------------------------------|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Embedding                    | $x_1$  | $x_2$  |
| Queries                      | $q_1$  | $q_2$  |
| Keys                         | $k_1$  | $k_2$  |
| Values                       | $v_1$  | $v_2$  |
| Score                        | $q_1 \cdot k_1 = 112$                                                                     | $q_1 \cdot k_2 = 96$                                                                      |
| Divide by 8 ( $\sqrt{d_k}$ ) | 14                                                                                        | 12                                                                                        |
| Softmax                      | 0.88                                                                                      | 0.12                                                                                      |

- The fifth step is to multiply each value vector by the softmax score (in preparation to sum them up)
- The sixth step is to sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).

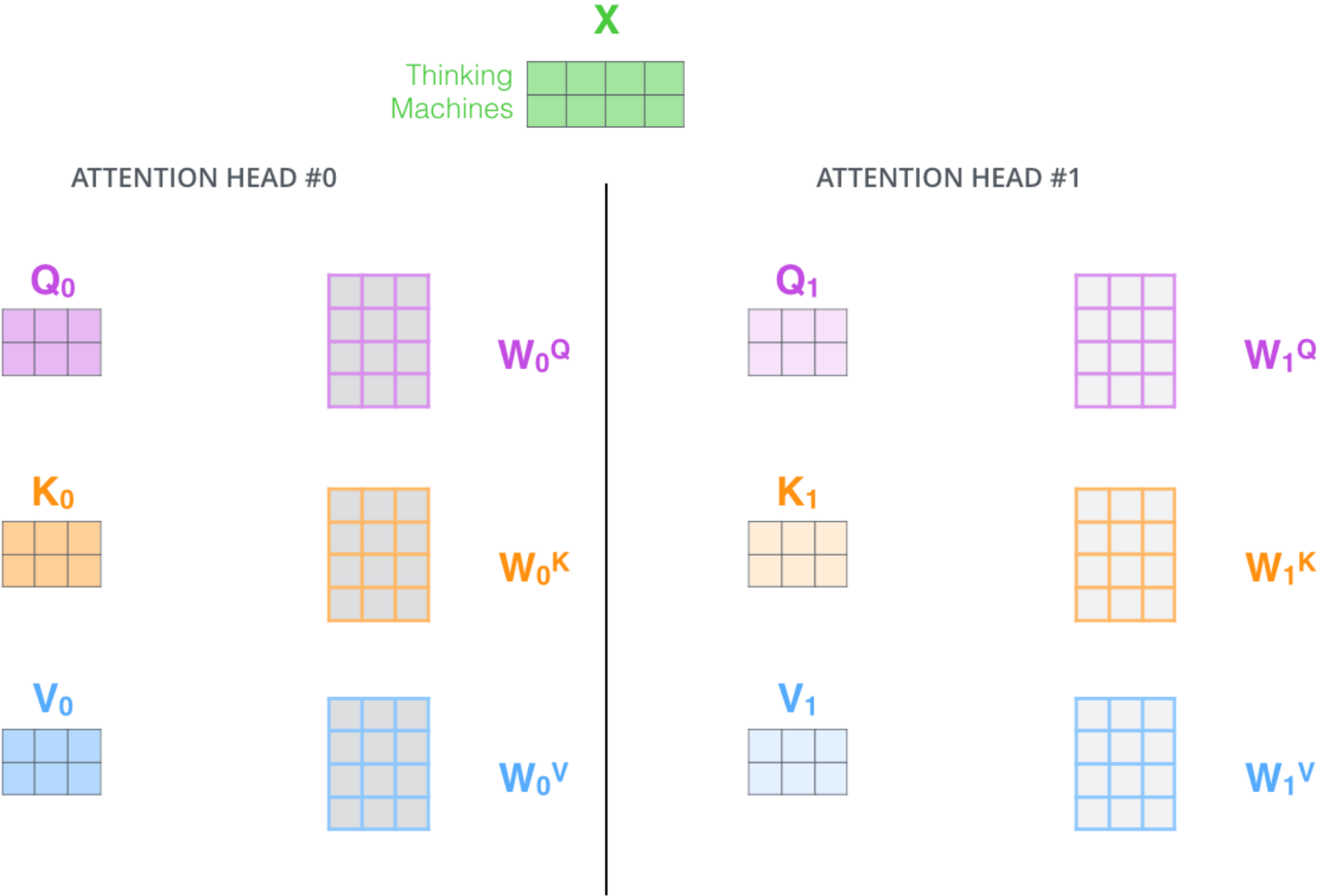




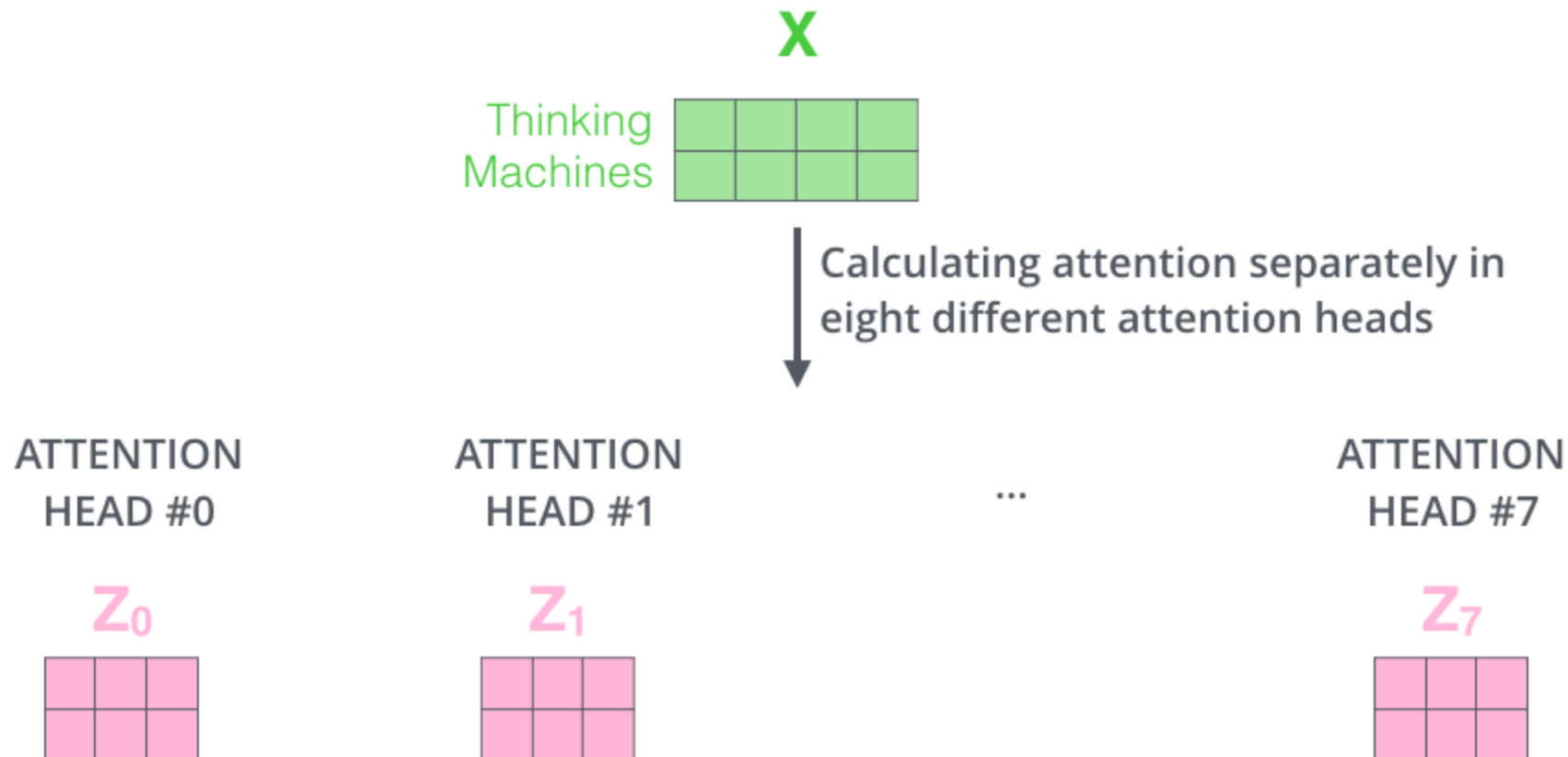
Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.

$$\text{softmax} \left( \frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$
$$= \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

“multi-headed” attention improves the performance of the attention layer

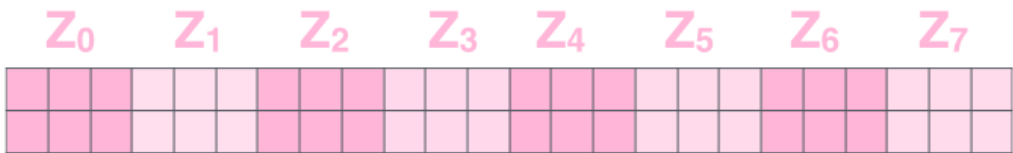


If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different  $Z$  matrices



Condense these eight down into a single matrix for feed-forward layer

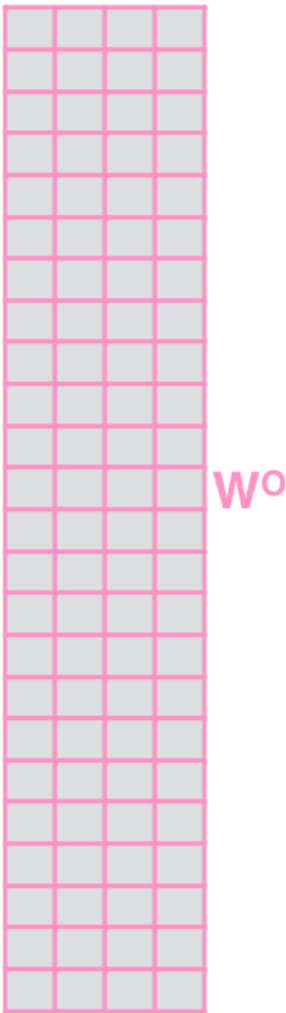
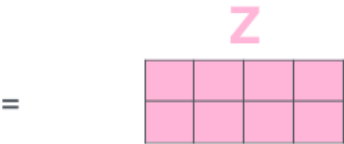
1) Concatenate all the attention heads



2) Multiply with a weight matrix  $W^O$  that was trained jointly with the model

$\times$

3) The result would be the  $Z$  matrix that captures information from all the attention heads. We can send this forward to the FFNN



1) This is our input sentence\*

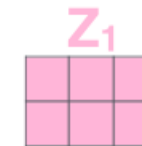
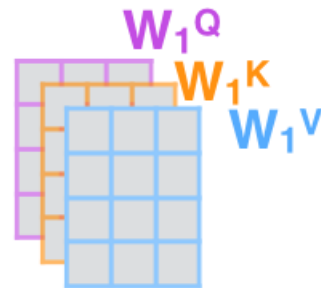
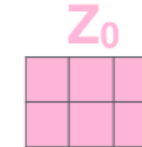
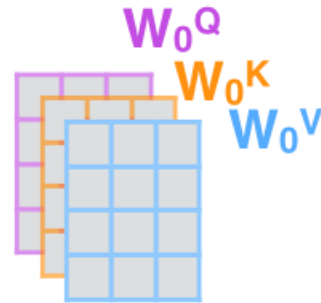
2) We embed each word\*

3) Split into 8 heads. We multiply  $X$  or  $R$  with weight matrices

4) Calculate attention using the resulting  $Q/K/V$  matrices

5) Concatenate the resulting  $Z$  matrices, then multiply with weight matrix  $W^O$  to produce the output of the layer

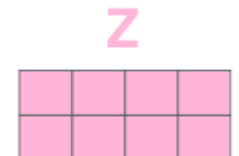
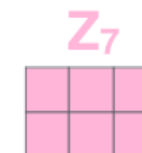
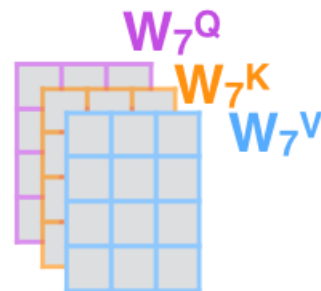
Thinking Machines



...

...

...

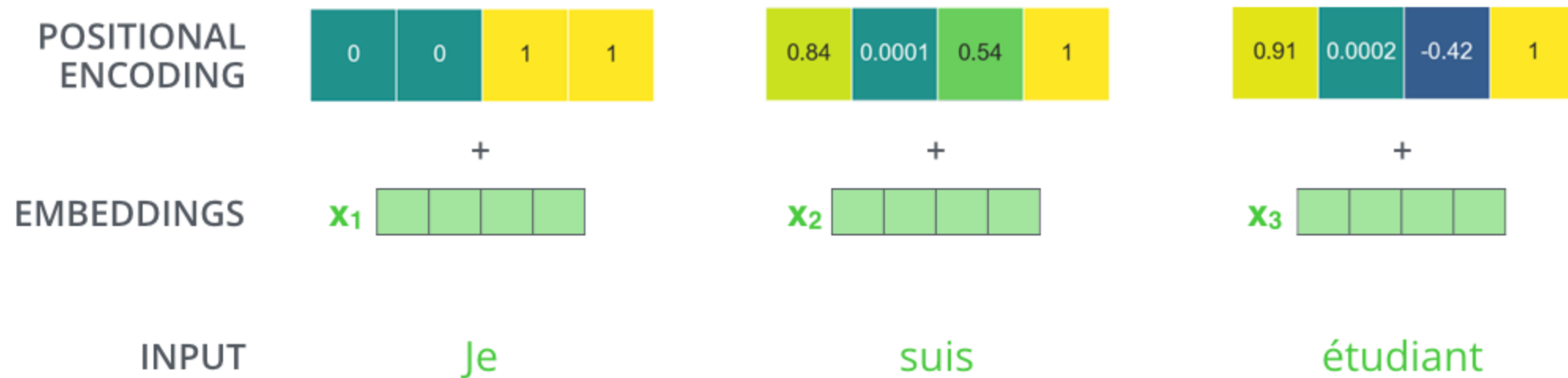


\* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

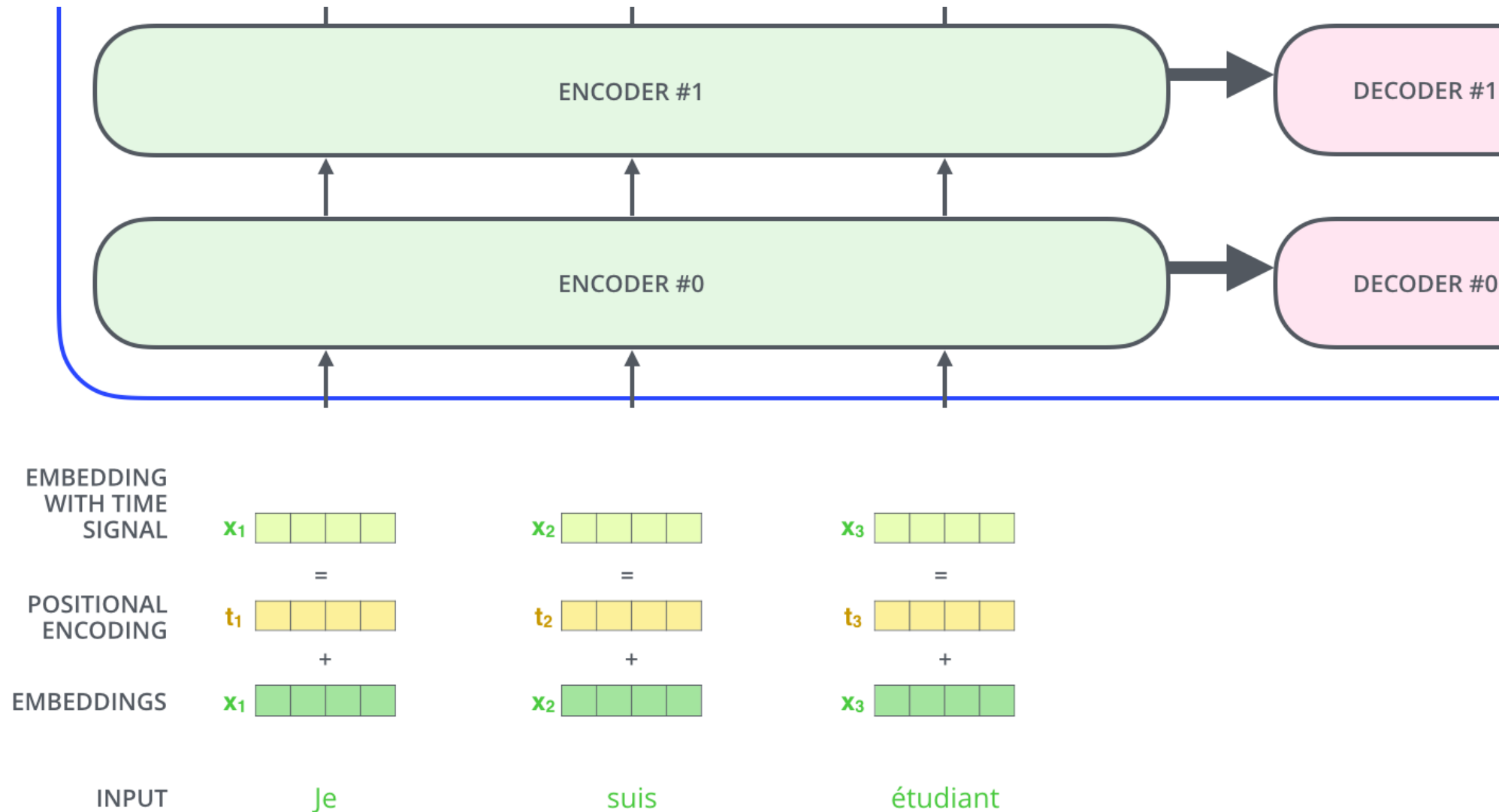


## Positional Encoding

To account for the order of the words in the input sequence, transformer adds a vector to each input embedding. These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence



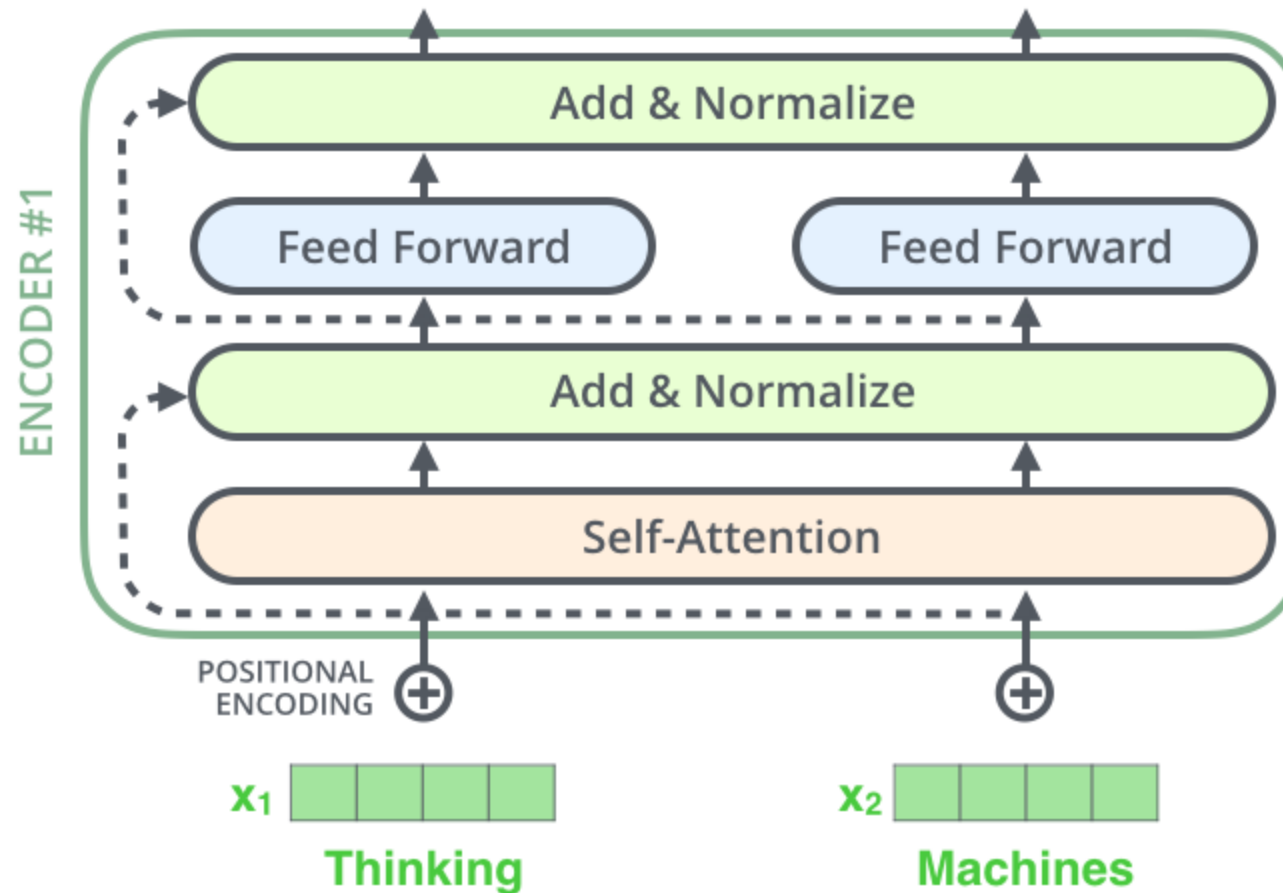
## Positional Encoding





## The Residuals – Add & Norm

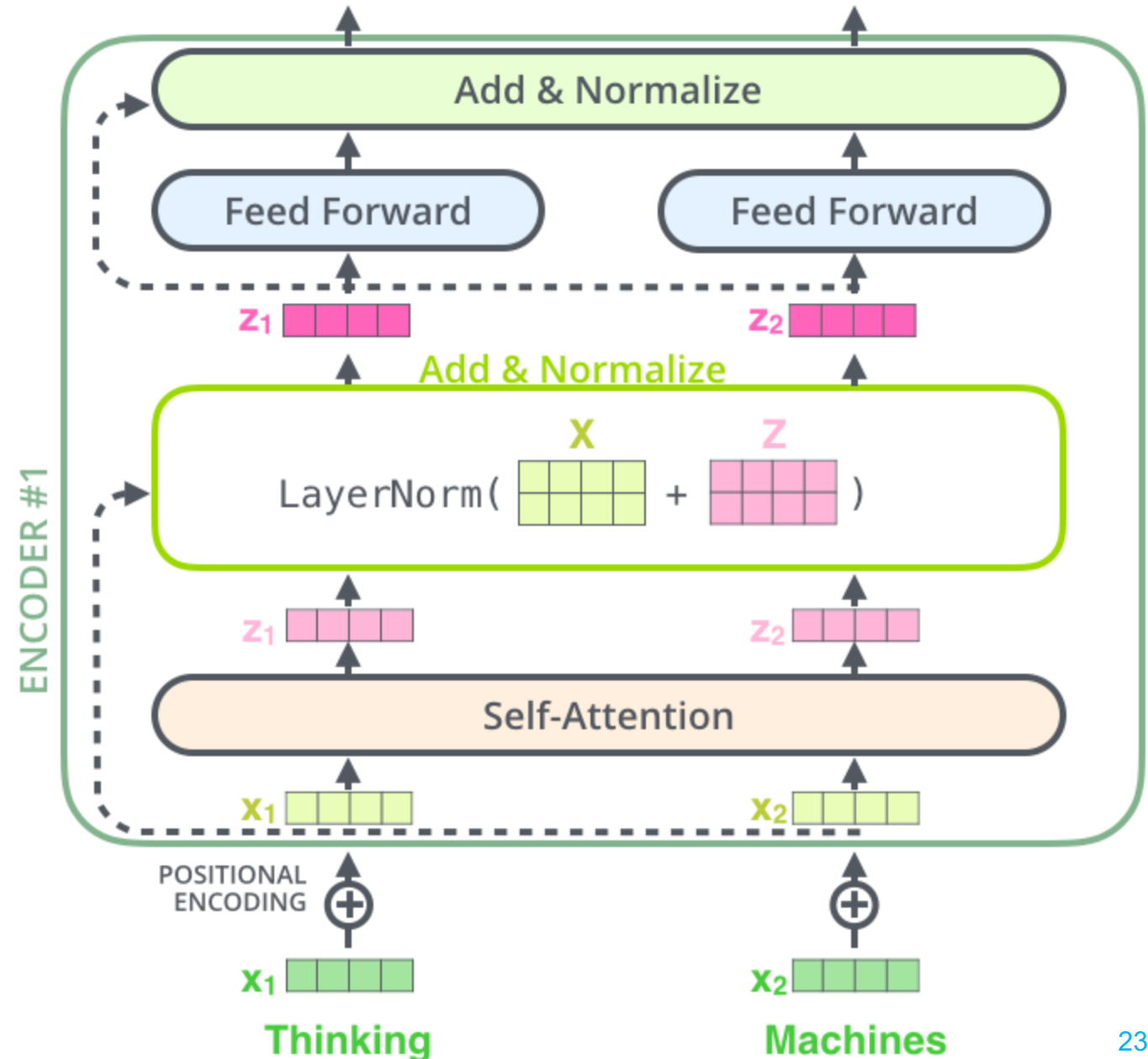
It allows information and gradient transitions to transfer directly from the previous layer to the next without transformation, enabling deep learning models to avoid "gradient vanishing".



## The Residuals – Add & Norm

$$x'' = \text{LayerNorm}(x' + \text{FeedForward}(x'))$$

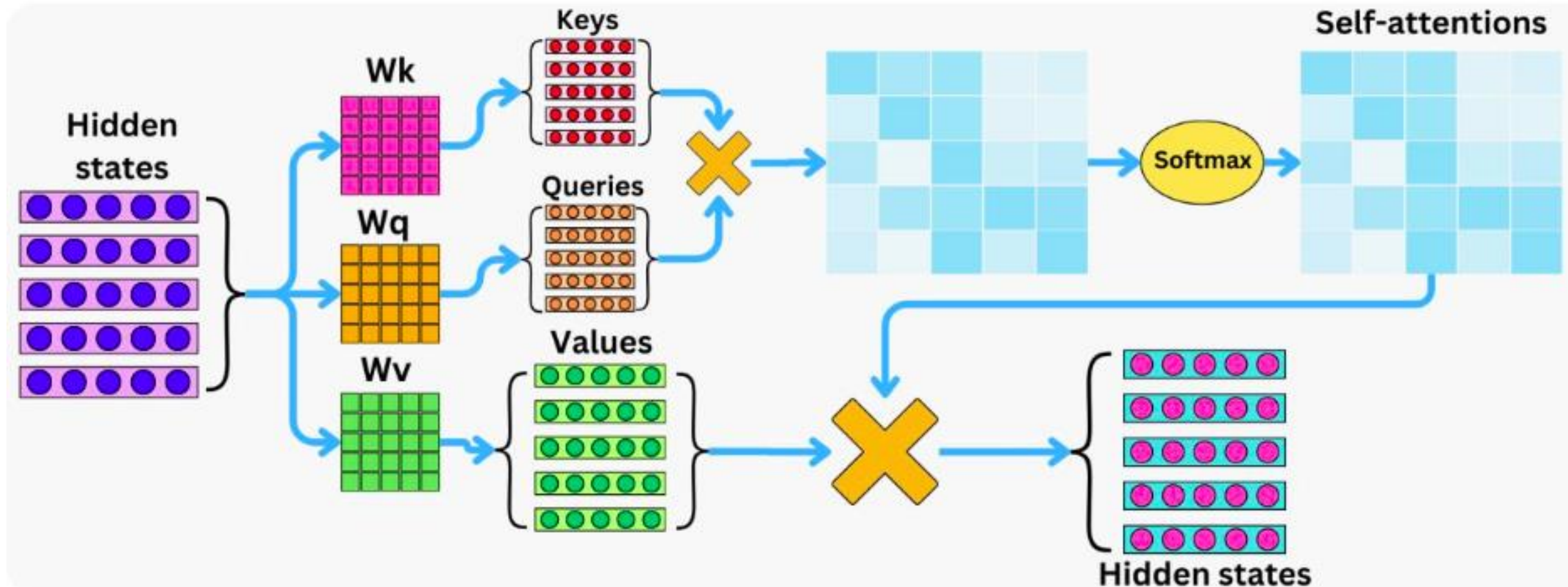
$$x' = \text{LayerNorm}(x + \text{MultiHeadAttention}(x))$$



## Self-Attention vs. Cross-Attention

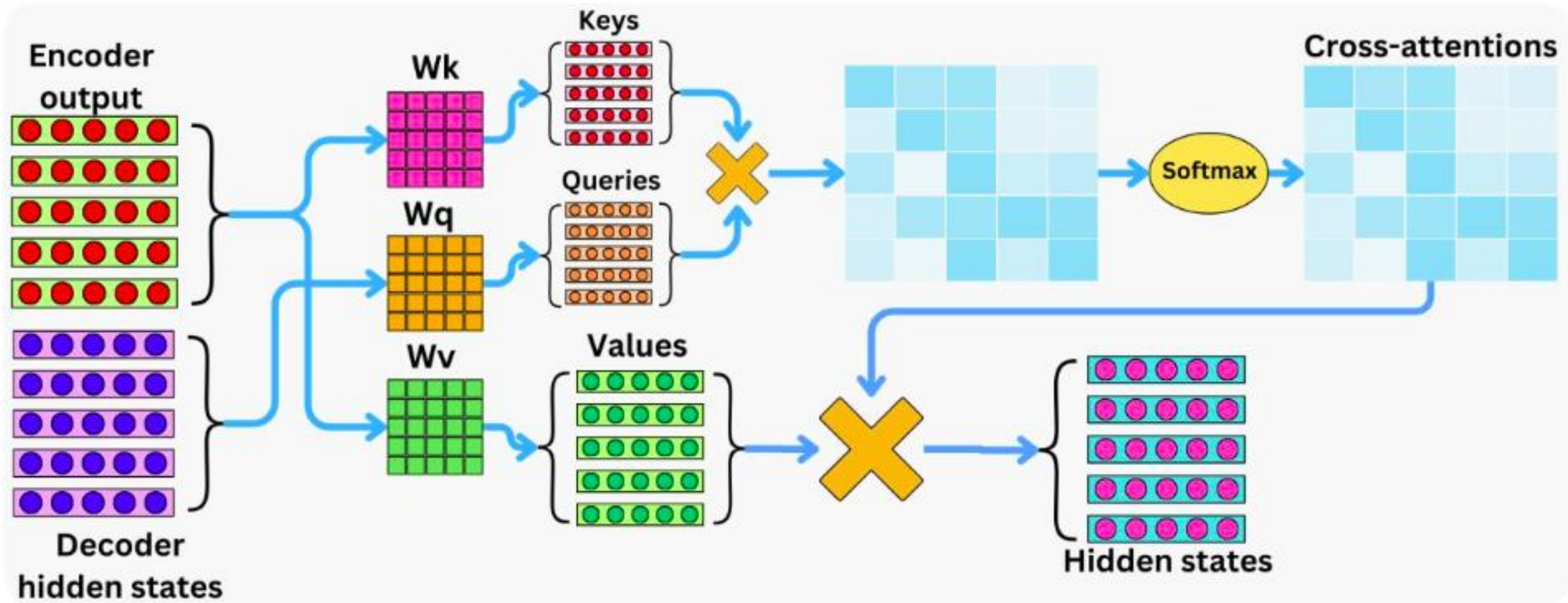
- **Self-Attention:**  $Q, K, V$  all come from the same source (e.g., within the encoder). computes the interactions between the different elements of an input sequence.  
*Eg:* the different words in a sentence.
- **Cross-Attention:**  $Q$  comes from the decoder, while  $K, V$  come from the encoder. Computes the interactions between the elements of 2 different input sequences
- *Eg:* how words in one sentence influence words of another sentence

# Self-Attention vs. Cross-Attention





# Self-Attention vs. Cross-Attention



| Aspect            | Cross-attention                                                                                                  | Self-attention                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Definition</b> | Allows the decoder to use information from the encoder.                                                          | Allows each word in a sequence to focus on other words in the same sequence.                   |
| <b>Uses</b>       | Used in tasks like translation, where the model must look at the entire input (encoder) while generating output. | Used to understand relationships within a single input (e.g. text) without external context.   |
| <b>Focus</b>      | Focuses on picking useful information from another part of the model.                                            | Focuses on how words in the same sentence relate to each other.                                |
| <b>Data Flow</b>  | Involves interaction between two different data parts (encoder and decoder).                                     | Data flows within the same sequence (internal attention).                                      |
| <b>Example</b>    | In translation, cross-attention helps the decoder choose the right words from the encoder.                       | In text analysis, self-attention helps the model understand how words in the sentence connect. |

## Conclusion and Summary

- Attention mimics human focus.
- Transformer relies on Self-Attention, eliminating recurrence.
- Key components: Multi-head attention, Positional encoding, Residual connections.